

How to ♥ LÖVE

Découvrir la programmation avec Lua

1 Premiers pas

Dans cette activité tu vas apprendre à programmer un jeu vidéo pas à pas grâce à LÖVE (Lua). C'est le langage utilisé pour créer certains jeux vidéos très connus comme Move or Die.



Pour commencer tu dois utiliser le bon vocabulaire pour que l'ordinateur te comprenne. Par exemple pour afficher quelque chose sur l'écran tu dois utiliser la fonction *draw* (dessiner en anglais).

```
function love.draw()  
    -- À l'intérieur de cette fonction on dessine tout ce qu'on veut  
end
```

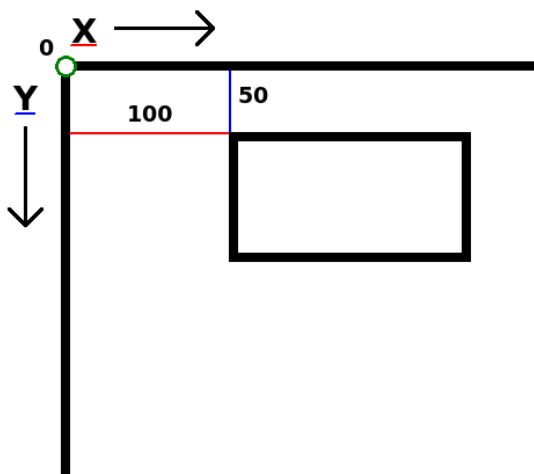
On va commencer par dessiner quelque chose de simple : un rectangle. À toi de jouer, recopie ces lignes de codes dans un nouveau fichier :

```
1 function love.draw()  
2     -- À l'intérieur de cette fonction on va dessiner un rectangle  
3     love.graphics.rectangle('line', 100, 50, 200, 150)  
4 end
```

Puis sauvegarde ton fichier sous le nom «main.lua» dans ton dossier personnel. Pour lancer ton jeu, il suffit de glisser-déposer ton dossier personnel sur l'icône ♥ de ton bureau.



Essaie de changer les différents nombres à l'intérieur de la parenthèse, puis explique à quoi servent chacun de ces nombres.



Les deux premiers nombres sont les coordonnées du rectangle, aide-toi du petit schéma ci-contre pour mieux comprendre.

2 Automatisation

Tu vas devoir modifier un peu ton fichier pour améliorer ton jeu :

```
1 x = 100
2 y = 50
3
4 function love.draw()
5     -- Dessine un rectangle en position (x,y)
6     love.graphics.rectangle('line', x, y, 200, 150)
7 end
```



Maintenant modifie les valeurs de x et y pour afficher le rectangle au milieu de l'écran.

Lorsqu'on change les valeurs de x et y , on modifie les coordonnées du rectangle sur l'écran. Nous allons apprendre comment le faire automatiquement grâce à LÖVE !

Pour cela il faut utiliser la fonction *update* (actualiser en anglais).

```
function love.update()
    -- À l'intérieur de cette fonction on actualise les informations
end
```

Ensuite il suffit de demander à LÖVE d'augmenter x petit à petit : rien de plus simple, ajoute ces lignes à la suite de ton fichier :

```
1 x = 100
2 y = 50
3
4 function love.draw()
5     -- Dessine un rectangle en position (x,y)
6     love.graphics.rectangle('line', x, y, 200, 150)
7 end
8
9 function love.update()
10    -- x augmente petit à petit
11    x = x + 5
12 end
```



Que ce passe-t-il ?

Essaie de programmer ton jeu pour que le rectangle se dirige tout seul vers le coin en bas à gauche de l'écran. Appelle le professeur lorsque tu as réussi.



Petit indice :

N'oublie pas que « x » permet de te déplacer horizontalement et « y » pour te déplacer verticalement

3 En avant les images

Pour ton jeu vidéo, il serait dommage de se contenter d'un simple rectangle... Choisis plutôt une belle image de voiture sur internet puis enregistre-la dans ton dossier. Il ne reste plus qu'à demander à LÖVE d'afficher ton image à la place du rectangle :

```
4 function love.draw()  
5   -- À l'intérieur de cette fonction on va afficher notre voiture  
6   love.graphics.draw(MaVoiture, x, y)  
7 end
```

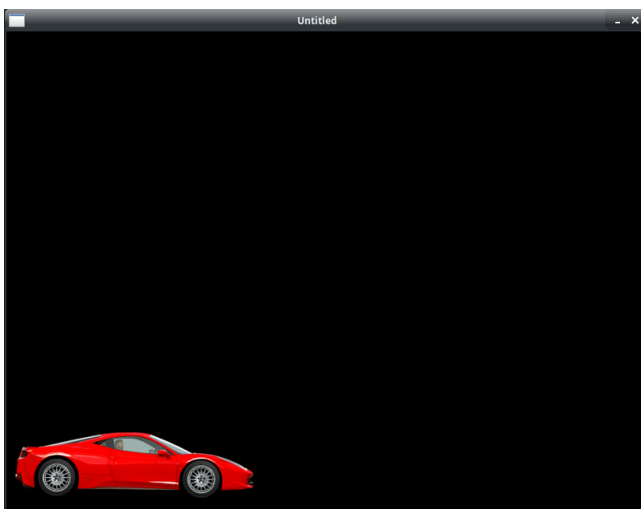


Remplace le rectangle par la voiture à l'intérieur de ton code.
Que se passe-t-il lorsque tu lances ton jeu vidéo ?

Ne t'inquiète pas c'est normal, il manque une étape pour afficher ton image, il faut d'abord charger ton image avant de l'afficher.

Pour cela tu vas ajouter au début de ton fichier :

```
1 -- On charge notre image «voiture.png» dans le jeu vidéo :  
2 MaVoiture = love.graphics.newImage('voiture.png')  
3 x = 100  
4 y = 50
```



Et voilà le tour est joué, il ne te reste plus qu'à modifier les valeurs de x et y pour afficher la voiture en bas à gauche de l'écran.



Une voiture ce n'est pas très original, si ça ne te plaît pas n'hésite pas à utiliser une autre image, voir même à en créer une toi-même !

4 Et les touches du clavier?

Dans un jeu vidéo il arrive que des objets se déplacent tout seul, mais on aimerait bien guider notre voiture avec les flèches du clavier. Ça tombe bien, il y a fonction pour ça (comme d'habitude) : c'est la fonction *keyboard* (clavier en anglais).

```
11 function love.update()  
12   -- on avance lorsque la touche «right» est enfoncée  
13   if love.keyboard.isDown('right') then  
14     x = x + 5  
15   end  
16 end
```



Recopie ces lignes à l'intérieur de la fonction *love.update()* puis complète ton code pour que la voiture recule lorsque tu appuies sur la touche «left».

Nous voilà déjà bien avancé, mais il y a encore un petit problème, que remarque-tu lorsque la voiture se déplace sur les côtés ?



Voici ce qu'a fait un élève pour empêcher la voiture de sortir de l'écran. Malheureusement son code ne fonctionne pas, à toi de corriger les erreurs ci-dessous :

```
11 function love.update()  
12   if love.keyboard.isDown('right') and x < 400 then  
13     Voiture.x = Voiture.x + 5  
14   if love.keyboard.isDown('left') and Voiture >      then  
15     Voiture.x = Voiture.x - 5  
16   end  
17 end
```



Bien sûr si tu en a l'envie tu peux faire déplacer ton bolide dans toutes les directions, mais n'oublie pas de l'empêcher de sortir de l'écran.

5 Ranger notre code

Pour ne pas que ton jeu devienne un vrai labyrinthe à coder il est très important de bien ranger les informations et ne pas se mélanger les pinceaux.

Pour cela on va rassembler et séparer les informations grâce à une méthode de programmation très pratique : tu vas créer une *classe Voiture* pour ranger toutes les informations qui concerne la voiture.

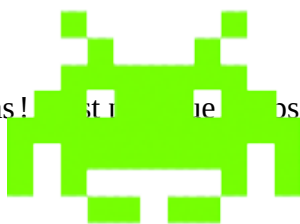
```
1 -- On créer une classe Voiture pour ranger les informations :
2 Voiture = {}
3 Voiture.image = love.graphics.newImage('images/ferrari.png')
4 Voiture.x = 50
5 Voiture.y = 500
6
7 function love.draw()
8     love.graphics.draw(Voiture.image, Voiture.x, Voiture.y)
9 end
10
11 function love.update()
12     if love.keyboard.isDown('right') and Voiture.x < 500 then
13         Voiture.x = Voiture.x + 5
14     elseif love.keyboard.isDown('left') and Voiture.x > 0 then
15         Voiture.x = Voiture.x - 5
16     end
17 end
```

Voilà ce sera maintenant beaucoup plus pratique pour s'y retrouver car on sait à qui appartient l'information : *Voiture.x*, *Voiture.image*, ...

Toutes les informations qui concerne une *classe* sont attachées a celle-ci en les séparant par un point : *Classe.information*

6 Space invader

Une voiture toute seule, ça ne suffit pas ! Il est temps de compliquer un peu le challenge.



En utilisant ce que tu as appris, tu dois afficher un alien en haut de l'écran et la faire se déplacer tout seul horizontalement en faisant des aller-retours.

 Petit indice :

tu devras introduire une nouvelle variable qui change de signe pour inverser le sens de l'alien.



Peut-être que tu as déjà deviné la suite ? C'est le moment de passer à l'action !
Fais en sorte que ton alien tire un laser, une bombe ou un sapin de Noël sur la voiture.



Tu dois remarquer qu'il y a un petit problème avec notre méthode de *classe* : on aimerait bien pouvoir tirer plusieurs rayon laser bien sûr ! As-tu une idée ?

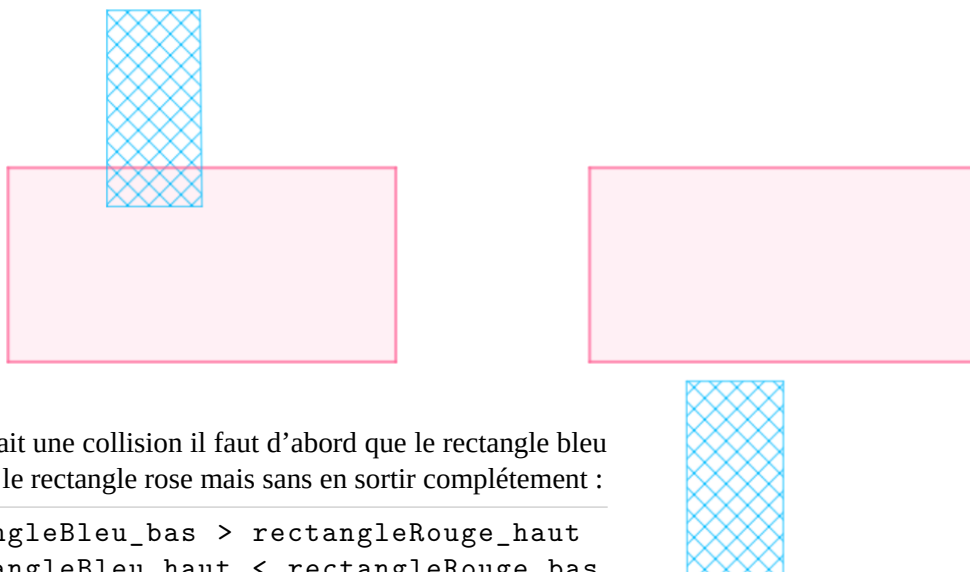
Eh bien il y a une solution assez simple : il suffit de créer nous-même une *function* qui va tirer le laser à chaque fois qu'on l'utilise :

```
function laserShot()  
  Laser = {}  
  Laser.image = love.graphics.newImage('images/laser.png')  
  Laser.x = Alien.x + 50  -- la position de départ du laser  
  Laser.y = Alien.y + 100 -- correspond à la position de l'alien  
end  
laserShot()  -- tire un seul laser  
  
-- Et on peut aussi tirer à volonté, par exemple :  
function love.update()  
  Laser.y = Laser.y + 7  
  if love.keyboard.isDown('space') then  
    laserShot()  
  end  
end
```

Enfin quelque chose qui secoue ! Mais pour que ça fonctionne complètement, tu vas devoir le programmer dans ton jeu :

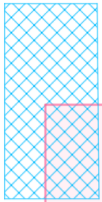
7 Hit hit hit box

L'idée est de vérifier si le laser touche la voiture, très simple en apparence mais il faut quand même réfléchir un peu ... Illustrons le problème grâce à deux rectangle pour mieux comprendre :



Pour qu'il y ait une collision il faut d'abord que le rectangle bleu pénètre dans le rectangle rose mais sans en sortir complètement :

```
if rectangleBleu_bas > rectangleRouge_haut  
and rectangleBleu_haut < rectangleRouge_bas
```

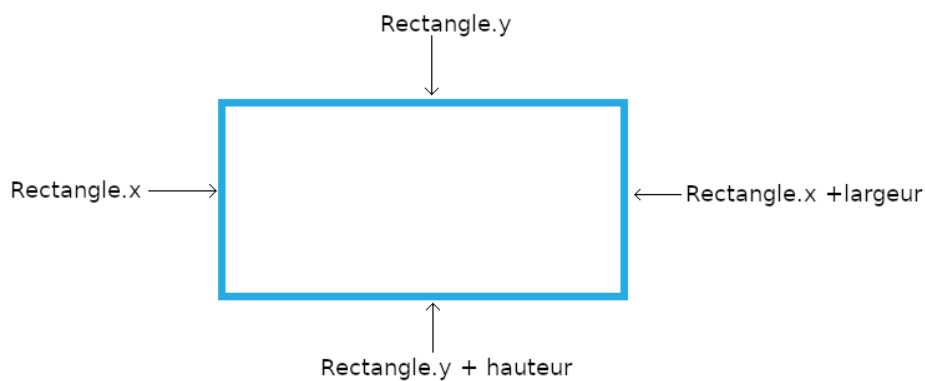


Il faut aussi que le côté droit du rectangle bleu soit entré à l'intérieur du rectangle rose mais sans en sortir (trop à droite) :

```
if rectangleBleu_bas > rectangleRose_haut
and rectangleBleu_haut < rectangleRose_bas
and rectangleBleu_droit > rectangleRose_gauche
and rectangleBleu_gauche < rectangleRose_droit
then ...
```



Maintenant que tu connais la théorie, essaie d'appliquer cela au laser et à la voiture. Pour cela, aide-toi du schéma ci-dessous :



Petit indice :

Pour faire un *game-over* tu peux utiliser la fonction : `love.event.quit('restart')`

8 [Prêt pour la suite](#)

Et voilà tu connais maintenant les bases pour programmer un jeu vidéo, Félicitations !

Il te reste encore de nombreuses choses à découvrir avant de créer ton propre jeu vidéo mais même la plus grande des aventures commence toujours par un premier pas.



